جلسهٔ ۴ از ۵

امنیت و ارزیابی عامل‌های کدنویسی شهودی

اعتماد مؤثر (Effective Trust): از هفت ستون امنیت تا سنجش کیفیت درون جعبهٔ شیشه‌ای

بر پایهٔ مقالهٔ سوکراتیس کارتاکیس، آرون آیدلمن، وفاء بکالی و ملتم سوباشی‌اوغلو

پل از جلسات پیشین

عامل را توانمند کردیم؛ حالا چه؟

جلسهٔ ۱
مهار (harness)

←

جلسهٔ ۲
ابزار (tool)

←

جلسهٔ ۳
دانش رویه‌ای

←

جلسهٔ ۴
چه چیزی خراب می‌شود و چطور می‌سنجیم؟

واقعیت تازه: ما دیگر فقط برنامهٔ وب را در برابر سوءاستفاده‌های سنتی امن نمی‌کنیم؛ باید **نیروی کاری غیرانسانی** را امن کنیم که اختیار (authority) محیطی دارد: کد تولیدشده را اجرا می‌کند، به APIهای داخلی حساس می‌رسد و محیط تولید را پویا (dynamic) تغییر می‌دهد.

اعتماد را باید در دو محور بازتعریف کرد

امنیت

به شما می‌گوید آیا عامل (agent) **درون مرز** ماند — یعنی ایمن و بدون قصد (intent) مخرب عمل کرد.

ارزیابی

به شما می‌گوید آیا آنچه **درون آن مرز** رخ داد، ارزش انتشار دارد.

و این دو مستقل‌اند: عاملی می‌تواند همهٔ وارسی‌های امنیتی را پاس کند و همچنان قصد توسعه‌دهنده را بنیاداً بد بفهمد، قراردادهای پروژه را نادیده بگیرد، یا تجربهٔ کاربر را بی‌سروصدا خراب کند.

چرا اعتماد صفر و یکی دیگر کار نمی‌کند

در نرم‌افزار قطعی

اعتماد صفر و یکی است: کد کامپایل می‌شود، تست‌ها (test) پاس می‌شوند، اعتبارنامهٔ ایستا معتبر است. اگر توکن (token) درست باشد، سامانه به مسیر (trajectory) اجرا اعتماد می‌کند.

در سامانهٔ عامل‌محور

عامل می‌تواند توکن معتبر داشته باشد و در عین حال با **قصد ناهم‌راستا** خودگردان (autonomous) عمل کند. هویت (identity) ایستا، محیط دفاعی ضعیفی است.

اعتماد مؤثر: اعتماد دیگر دروازه‌ای نیست که عامل یک‌بار هنگام دیپلوی (deploy) از آن عبور کند؛ باید پیوسته کسب، راستی‌آزمایی و بر پایهٔ زمینهٔ رانتایم (runtime) اعمال شود — سنجه‌ای پیوسته در چهار محور: زنجیرهٔ تأمین، هویت، رفتار رانتایم، و پیوندهای زمینه‌ای.

از «هویت به‌مثابه محیط» به «زمینه به‌مثابه محیط»

در محیط سنتی سازمانی، دسترسی با مدل‌های ایستای مبتنی بر نقش اداره می‌شود. اما چون خطاهای روزمرهٔ عامل اغلب ریشه در ابزاری دارند که وجود ندارد، قاعده‌ای که مبهم تعریف شده یا گاردریلی که اصلاً تعریف نشده، سازمان‌ها باید به مدل زمینه‌محور کوچ کنند.

فرض بنیادی: باید فرض کنیم مدل زیرین ممکن است شکست بخورد یا به خطر بیفتد. پس امنیت نمی‌تواند صرفاً درون خودِ هوش مصنوعی ساکن باشد؛ باید یک **پاکت ایمنی بیرونی** و سخت‌گیرانه بر آن تحمیل کنیم.

بخش یک

هفت ستون معماری امنیت عامل

مبنای اجباری که مهار امن هر سامانهٔ خودگردان را می‌سازد.

ستون‌های ۱ و ۲ — زیرساخت و داده

۱. زیرساخت و شبکه

مالک: مهندسان زیرساخت ابری. **تهدید:** آلودگی بالادست و فرار از کانتینر (container).

اجرای رانتایم را در سندباکس‌های (sandbox) زودگذر (ephemeral) و سطح هسته ایزوله کنید. حکمرانی سخت‌گیرانهٔ خروج شبکه تضمین می‌کند دادهٔ تولیدشده فقط از مسیرهای مجاز — کش (cache) برون‌خط یا پروکسی داخلی صریح — عبور کند.

۲. داده

مالک: معماران داده. **تهدید:** نشت (leak) اطلاعات حساس از کانتکست ویندو، و بلعیدن دادهٔ آلوده.

دادهٔ در حال سکون با کلیدهای مدیریت‌شدهٔ مشتری و دادهٔ در حال انتقال با mTLS محافظت می‌شود. و مهم: انبارهای حافظهٔ بلندمدت — به‌ویژه پایگاه‌های بُرداری — باید افراز سخت‌گیرانهٔ مستأجران را اعمال کنند تا پیلود (payload) مخربی که یک مستأجر وارد کرده، در جست‌وجوی شباهتِ مستأجر دیگر بازیابی (retrieval) نشود.

ستون‌های ۳ و ۴ — مدل و برنامه

۳. مدل

مالک: مهندسان هوش مصنوعی. **تهدید:** حمله‌های معنایی که دستورهای مدل را واژگون می‌کنند.

در گردش‌کار (workflow) عامل‌محور، پرامپت و «فایل‌های دستور (instruction) و قاعده» که می‌گویند عامل از چه کاری منع شده، **کد منبع تازه** هستند. پس باید مثل مصنوع‌های بسیار حساس و گواهی‌شدهٔ رمزنگاشتی با آن‌ها رفتار شود.

۴. برنامه و رانتایم

مالک: توسعه‌دهندگان عامل. **تهدید:** خودگردانی مهارنشده هنگام اجرای منطق و استفاده از ابزار.

چون عامل‌ها تعاملی کار می‌کنند، دیوارهٔ آتش مبتنی بر قاعدهٔ سنتی کافی نیست: دیوارهٔ آتش مدل‌زبانی برای پالایش پویا، به‌همراه قلاب‌های (hook) قطعی (deterministic) در نقاط مشخص چرخهٔ عمر. دروازهٔ متمرکز عامل، نقش نگهبان اکوسیستم را بازی می‌کند.

ستون ۵ — هویت و مدیریت دسترسی

مالک: مدیران هویت. **خطر اصلی:** مسئلهٔ «معاون گیج (Confused Deputy)» — عاملی با امتیاز بیش‌ازحد که فریب می‌خورد و فرمان غیرمجاز اجرا می‌کند.

هویت یکتای رمزنگاشتی

به هر عامل هویت منحصربه‌فرد و قابل‌راستی‌آزمایی بدهید.

کنترل دسترسی مبتنی بر ویژگی

مجوز بر پایهٔ ماتریس **قصد × کاربر × زمان**، نه صرفاً نقش.

کاهش دامنهٔ به‌هنگام

اعتبارنامهٔ تازه و به‌شدت محدود که دقیقاً با پایان کار منقضی می‌شود.

ستون‌های ۶ و ۷ — رصدپذیری و حاکمیت

۶. رصدپذیری و عملیات امنیتی

تهدید: «خطاهای نامرئی» — عاملی که بی‌سروصدا وارد حلقهٔ استدلال بی‌پایان می‌شود.

بدون گزارش، ردیابی و اندازه‌گیری، هیچ راهی نیست بفهمیم عامل خوب کار می‌کند یا آرام منحرف شده. راه‌حل: دیپلوی سه‌گانهٔ خودگردان عملیات امنیتی (قرمز، آبی، سبز).

۷. حاکمیت

تهدید: تصمیم‌های خودگردانی که استانداردهای مقرراتی را برآورده نمی‌کنند.

نیازمند نظارت پیوسته، اولویت‌بندی مخاطره (کدام گردش‌کار خودگردان بیشترین اثر کسب‌وکاری را در صورت به‌خطر افتادن دارد؟) و ردِ ممیزی تغییرناپذیر که هر کنش واقعی را به عاملی مشخص و انسانی که مستقر یا تأییدش کرده نسبت می‌دهد.

و یک تصحیح مهم: اگر کد تولیدشده بدون خطا کامپایل شود، توسعه‌دهنده معمولاً فرض می‌کند امن است. راه‌حل: جایگزین‌کردن دکمه‌های سادهٔ تأیید با **«بازبینی منطق»** اجباری که نحو پیچیده را به زبان ساده برمی‌گرداند.

هفت ستون در یک نگاه

ستون	مالک سازمانی	تهدید اصلی
۱. زیرساخت و شبکه	مهندسی زیرساخت ابری	آلودگی بالادست، فرار از کانتینر، خروج ناخواستهٔ داده
۲. داده	معماری داده	نشت زمینه، بلعیدن دادهٔ آلوده، آلودگی بُرداری میان‌مستأجری
۳. مدل	مهندسی هوش مصنوعی	حملهٔ معنایی و واژگونی دستورها
۴. برنامه و رانتایم	توسعهٔ عامل	خودگردانی مهارنشده، حرکت جانبی غیرمجاز
۵. هویت و دسترسی	مدیریت هویت	معاون گیج، اختیار محیطی بیش‌ازحد
۶. رصدپذیری (observability) و عملیات	عملیات امنیت	خطای نامرئی، انحراف (drift) خاموش
۷. حاکمیت	حاکمیت و انطباق	مسئولیت حقوقی تصمیم خودکار، نبود رد ممیزی

بخش دو

سندباکس و دفاع زنجیرهٔ تأمین

عامل‌های شهودی به‌ندرت در نخستین تلاش کد بی‌نقص می‌نویسند — و همین، هستهٔ مسئله است.

حلقهٔ شهودی و چرا باید ایزوله شود

می‌نویسد
اسکریپت تولید می‌کند
←
اجرا می‌کند
و خطا می‌گیرد
←
می‌خواند
گزارش خطا را
←
بازنویسی می‌کند
تا با خواستهٔ کاربر بخواند

چون این فرایند مولد، تغییرپذیری بالایی وارد می‌کند، کد حاصل را نمی‌توان به‌طور ضمنی معتبر دانست. اجرای این اسکریپت‌ها در کنار عامل ریشه یا روی زیرساخت میزبان استاندارد، سطح ریسکی غیرقابل‌قبول می‌سازد.

سندباکس فقط زندان پیلود مخرب نیست: باید فعالانه دسترسی خام به میزبان را مسدود کند و **وضعیتش را میان اجراها کاملاً بازنشانی کند** — تا حتی اگر اسکریپتی آسیب‌پذیر یا دستکاری‌شده باشد، منطق به‌خطرافتاده نتواند ماندگار شود.

بسته‌های توهمی: آسیب‌پذیری ویژهٔ عصر عامل

مدل‌های زبانی مرتب بسته‌های نرم‌افزاری‌ای را نام می‌برند که وجود ندارند.

مهاجم چه می‌کند

انجمن‌های توسعه‌دهندگان و خروجی‌های هوش مصنوعی را برای یافتن این توهم‌ها (hallucination) رصد می‌کند و پیشاپیش بدافزار را با **دقیقاً همان نام‌های ساختگی** منتشر می‌کند.

چرا خطرناک‌تر از قبل است

چون عامل‌های خودگردان می‌توانند گراف وابستگی (dependency) را **بدون تأیید انسان** تغییر دهند، یک توهم کافی است تا بدافزار مستقیم وارد محیط ساخت شود.

دفاع سه‌لایه: وابستگی‌ها فقط از تأمین‌کنندگان بررسی‌شده یا فهرست‌های داخلی سازمان بیایند · قفل نسخهٔ رمزنگاشتی سخت‌گیرانه · و خط لوله پیش از هر ارتقا به تولید، صورت‌ریز اجزای نرم‌افزار و امضاهای دیجیتال را خودکار راستی‌آزمایی کند.

حکمرانی خروج شبکه

در نرم‌افزار سنتی ترافیک خروجی کاملاً قابل‌پیش‌بینی است. در سامانهٔ شهودی، خروج غیرقطعی (not deterministic) است، چون با استفادهٔ پویا از ابزارهای تازه‌ساخته هدایت می‌شود.

و فهرست مجاز کافی نیست: فهرست دامنه‌های تأییدشده نمی‌تواند عامل را در برابر **تزریق غیرمستقیم پرامپت** که در صفحه‌های وب شخص ثالث پنهان شده محافظت کند.

راه‌حل: دسترسی **غیرتعاملی** به اینترنت. عامل باید اطلاعات بیرونی را منحصراً از راه کش‌ها برون‌خط یا سرویس‌های خزش وب اختصاصی و پیش‌پالایش‌شده بگیرد. با اجبار همهٔ داده به عبور از مسیرهای حکمرانی‌شده، عامل هرگز مستقیم با پیلود مخرب تعامل نمی‌کند.

دو الگوی خطا کد تولیدشده

۱. اعتماد بیش‌ازحد به مرورگر

تولید خودکار معمولاً کم‌مقاومت‌ترین راه را می‌رود و عملیات حساس را در سمت کاربر انجام می‌دهد: کلیدهای API، اعتبارسنجی گذرواژه و پرچم‌های نشست، مستقیم در سمت مرورگر رها می‌شوند.

یعنی هرکس ابزار توسعهٔ مرورگر را باز کند، می‌تواند اعتبارنامه‌ها (credential) را بخواند یا سطح دسترسی خودش را دست‌کاری کند — بدون هیچ گذرواژهٔ واقعی.

۲. پشتیبان رهاشده

سرعت ساخت از سرعت برپاکردن لایه‌های نامرئی امنیت جلو می‌زند. ابزارها در وصل‌کردن پایگاه داده یا بالاآوردن داشبورد مدیریت عالی‌اند، اما به‌ندرت کنترل‌های «رد پیش‌فرض» را فعال می‌کنند.

نتیجه: چیزهای پایه‌ای مثل امنیت سطح ردیف از قلم می‌افتند و دادهٔ خصوصی کاربران و محیط‌های داخلی به روی اینترنت باز می‌مانند.

ریشهٔ مشترک: کاربران کد تولیدشده را فقط به این دلیل که **کامپایل می‌شود و اجرا می‌شود** ضمناً معتبر می‌دانند — و همین آن‌ها را نسبت به دورزدن کامل کنترل‌های استاندارد پشتیبان کور می‌کند.

اصطکاک ویرایشگر در برابر اعمال در خط لوله

چرا مسدودسازی سخت در ویرایشگر جواب نمی‌دهد

به‌آسانی دور زده می‌شود و برای توسعه‌دهندهٔ بی‌خطری که دارد روی منطق پیچیده تکرار می‌کند، اصطکاک بیش‌ازحد می‌سازد.

تقسیم کار درست

در ویرایشگر: **لینترهای مشورتی** که راهنمایی زمان‌واقعی می‌دهند. در خط لوله: **اعمال قطعی و بی‌اغماض** با تست امنیت ایستا و تحلیل ترکیب نرم‌افزار.

قاعده: «انتقال به چپ» یعنی راهنمایی زودهنگام، نه دروازهٔ زودهنگام. دروازهٔ واقعی جایی است که نمی‌شود دورش زد.

جعل سرور ابزار و مجوزدهی زمینه‌ای

عامل‌ها در رانتایم سرورهای بیرونی یا داخلی را کشف و به آن‌ها وصل می‌شوند. این یک بردار تهدید حیاتی می‌سازد: سروری جعلی یا به‌خطرافتاده می‌تواند خود را ابزاری مشروع جا بزند تا پیلود مخرب تزریق کند یا امتیاز بیش‌ازحد بخواهد.

و چون عامل‌ها خودگردان‌اند، ممکن است فرمان‌های مخرب این سرورهای جعلی را **پیش از هر مداخلهٔ انسانی** اجرا کنند.

دیوارهٔ آتش مدل‌زبانی

در رانتایم و پیش از عامل فعال، تزریق‌های فرصت‌طلبانهٔ پرامپت را پویا رهگیری (interception) می‌کند.

دروازهٔ متمرکز عامل

مجوزدهی زمینه‌ای را ارزیابی (evaluation) می‌کند: آیا درخواست عامل برای صداکردن این ابزار، دقیقاً با قصد اولیهٔ توسعه‌دهنده هم‌راستاست؟ همهٔ فراخوان‌ها از این نقطهٔ ورود و خروج حکمرانی‌شده عبور می‌کنند.

معاون گیج: هویت تفویضی در برابر هویت عامل‌محور

مسئله وقتی رخ می‌دهد که تزریق پرامپت (prompt) — مثلاً دستوری مخرب پنهان در ریپوی متن‌باز که توسعه‌دهنده ناآگاهانه در کانتکست ویندواش چسبانده — عاملی با امتیاز بیش‌ازحد را فریب می‌دهد تا فرمان غیرمجازی را به نیابت مهاجم اجرا کند.

ضدالگو

عامل زیر اعتبارنامهٔ **تفویض‌شدهٔ کاربر انسانی** کار کند. این به عامل دسترسی محیطیِ خطرناک می‌دهد.

الگوی درست

عامل با هویتی اختصاصی که صریحاً **عامل‌محور** برچسب خورده احراز هویت کند. هویت متمایز و قابل‌مشاهده تضمین می‌کند مجوزهایش محدود و مشمول گزارش ممیزی دقیق بماند.

اصل بنیادی: عامل هرگز نباید **داور (judge) نهایی دسترسی** باشد.

اختیار محیطی صفر

عاملی که یک «حس‌وحال» را اجرا می‌کند، هرگز نباید امتیازهای کامل و محیطی مدیریتی توسعه‌دهنده را به ارث ببرد.

- **کاهش دامنهٔ به‌هنگام (JIT downscoping) توکن:** وقتی عامل اسکریپت یا مهارتی تازه می‌نویسد، سندباکس اجرا اعتبارنامه‌ای تازه و به‌شدت محدود می‌گیرد که دقیقاً به همان منابع دادهٔ لازم برای همان اسکریپت محدود است — نه ارث‌بردن مجوزهای گستردهٔ عامل والد.
- **فهرست مجاز درخت فایل:** عملیات خواندن و نوشتن به دایرکتوری‌های مشخص پروژه محدود شود، با قواعد **رد پیش‌فرض** برای مسدودکردن دسترسی به رازها، اسکریپت‌های ساخت و مانیفست‌های تولید.
- **انقضای فوری:** این توکن‌های کاهش‌یافته به‌شدت زودگذرند و دقیقاً همان لحظه‌ای که کار تمام می‌شود منقضی می‌شوند.

کنش‌های پرمخاطره: چرا دکمهٔ تأیید کافی نیست

مغالطهٔ «کار می‌کند، بفرستش»: کدنویسان شهودی اغلب برای نوشتن نحو پیچیده‌ای که کاملاً نمی‌فهمند به هوش مصنوعی تکیه می‌کنند. دروازه‌های (gate) سادهٔ تأیید سریع به **خستگی تأیید (Approval fatigue)** می‌انجامند و توسعه‌دهنده کورکورانه کدی را مجاز می‌کند که درکش نمی‌کند.

احراز هویت چندعاملی سخت‌افزاری

برای کنش‌های پرمخاطره، چالش فیزیکی اجباری: توسعه‌دهنده باید کلید امنیتی سخت‌افزاری را لمس کند تا اجرا را رمزنگاشتی تأیید کند.

تفاوت‌نمای شهودی

پیش از اجرای ابزار حیاتی، حدنصابی از ارزیاب‌ها درخواست را رهگیری می‌کنند و کد پیچیدهٔ تولیدشده را به خلاصه‌ای به زبان ساده برمی‌گردانند: «قصد مبهم اولیهٔ شما دقیقاً به این گام‌های اجرایی نگاشت می‌شود.»

بخش سه

سه‌گانهٔ قرمز، آبی و سبز

وقتی منطق برنامه با سرعتی بی‌سابقه تولید، اجرا و دور ریخته می‌شود، عملیات امنیتیِ دستی نمی‌تواند همپا بماند. خودِ عملیات امنیتی باید عامل‌محور شود.

پیلودهای مخرب نامرئی و آلودگی ریپو

خودِ ریپوها بردار حملهٔ بسیار مؤثری‌اند. مهاجم می‌تواند نویسه‌های یونیکد با عرض صفر یا نویسه‌های هم‌ریخت را مستقیم در کدبیس تزریق کند.

> این پیلودهای مخرب نامرئی در دید آشکار پنهان می‌شوند و از بازبینی انسانی می‌گریزند.

و چرا در عصر عامل بدتر است: چون عامل‌ها کد را بسیار سریع‌تر از انسان دست‌کاری و تکثیر می‌کنند، یک پیلود مخرب پنهان می‌تواند **ظرف چند دقیقه در صدها فایل** پخش شود، پیش از آنکه کسی متوجه شود.

تیم قرمز – عامل مهاجم

پایش منفعل بنیاداً واکنشی است. برای کشف آسیب‌پذیری‌های معنایی و بارهای نامرئی پیش از دشمن بیرونی، باید عامل‌های تیم قرمز مجازی مستقر کرد.

به‌جای تست نفوذ ایستا، عامل مهاجم فعالانه **«حس‌وحال خصمانه»** تزریق می‌کند.

- ساخت پویای فرارهای پیچیده به‌شکل ایفای نقش.
- دفن دستورهای مخرب پنهان درون بلوک‌های عظیم زمینهٔ بازیابی‌شده یا پست‌های ساختگی انجمن – همان‌هایی که توسعه‌دهندگان مرتب در ویرایشگرشان می‌چسبانند.
- آزمودن اینکه آیا عامل هدف با زمینهٔ آلوده منحرف می‌شود و راه‌حلی ناامن توهم می‌کند یا نه.

تیم آبی — عامل مدافع

تحلیل رفتاری کاربر و موجودیت سنتی برای هوش مصنوعی غیرقطعی بی‌اثر است. جایگزین: **تحلیل رفتاری عامل** (ABA) که مسیرهای اجرای مورد انتظار را مبنا (baseline) می‌گیرد و ناهنجاری‌های ویژهٔ عامل را تشخیص می‌دهد.

صورت‌ریز اجزای رانتایم (AgBOM): سیاهه‌ای پویا از ابزارها، مدل‌ها و منابع دادهٔ فعال عامل در هر میلی‌ثانیه. اگر منطق عامل منحرف شود — مثلاً اسکریپتی ناگهان شروع به پرس‌وجوی تعداد غیرعادی ابزار بیرونی کند یا وارد حلقهٔ منابع بی‌کران (Unbounded) شود — موتور تحلیل فوری انحراف را علامت می‌زند.

تیم سبز — عامل ترمیم‌گر

چرا روش سنتی بد است

کشتن خشن کانتینر میزبان، بسیار مخل و خطرناک است. پایان‌دادن به عامل در میانهٔ تفکر می‌تواند APIهای متصل را در وضعیت خراب رها کند.

قرنطینهٔ باحالت

دسترسی ابزار عامل با ظرافت سلب می‌شود و توانایی کنشش روی جهان منجمد می‌گردد، در حالی که **حافظهٔ کوتاه‌مدتش کاملاً دست‌نخورده** برای تحلیل قانونی باقی می‌ماند.

و یک گام فراتر — بازسازی خودکار: عامل ترمیم‌گر اسکریپت ناامن را خودگردان بازنویسی می‌کند تا آسیب‌پذیری را وصله کند، و سپس کد امنِ جایگزین را مستقیم در ویرایشگر به توسعه‌دهنده نشان می‌دهد — بدون نیاز به مداخلهٔ دستی برای فرمول‌بندی راه‌حل.

یکپارچه‌سازی سه‌گانه و دسته‌های کوچک

پیش‌شرط: برای جلوگیری از تولید تغییرهای عظیم و بازبینی‌ناپذیر، خروجی عامل را به **دسته‌های کوچک** محدود کنید. بهترین راه، حلقهٔ تست‌محور است که سامانه مانع می‌شود عامل هم‌زمان تست و کد پیاده‌سازی را تغییر دهد — تا تست، مبنای عینی بماند.

فاز	چه اتفاقی می‌افتد
برنامه‌ریز	وقتی عامل اصلی گردش‌کاری طراحی می‌کند، مهارتی تخصصی برای مدل‌سازی تهدید کمکش می‌کند نقص‌های منطقی و نقض سیاست (policy) را **پیش از شروع اجرا** شناسایی کند.
ارزیاب	حدنصاب ارزیاب‌ها رد اجرای پیشنهادی را بازبینی می‌کند، در حالی که عامل مدافع هم‌زمان صورت‌ریز اجزا را راستی‌آزمایی و زمینهٔ معنایی را برای انحراف قصد (Intent Drift) پایش می‌کند.
مجری	هنگام اجرای کنش کاهش‌دامنه‌یافته، عامل ترمیم‌گر اجرای واقعی ابزار را پایش می‌کند و آمادهٔ فعال‌کردن فوری قرنطینهٔ باحالت (Stateful quarantine) یا حلقهٔ بازسازی خودکار است.

بخش چهار

رصدپذیری: ممیزی ذهن عامل

قاعدهٔ بنیادی: چیزی را که نمی‌بینید نمی‌توانید امن کنید.

وقتی «موفقیت» دروغ می‌گوید

در ریزسرویس‌های سنتی، پاسخ موفق یعنی عملیات موفق. اما در سامانهٔ عامل‌محور، وضعیت «موفق» ممکن است صرفاً پوششی باشد بر سناریویی که در آن منطق درونی عامل بی‌سروصدا وارد حلقهٔ توهم شده است.

حملهٔ انکار کیف پول (Denial of Wallet): مهاجم عمداً حلقه‌های بی‌پایان و پرهزینهٔ محاسباتی را راه می‌اندازد تا صورت‌حساب ابری و مدل‌زبانی سازمان را عمداً منفجر کند. رصدپذیری دیگر فقط دغدغه‌ای عملیاتی برای در دسترس‌بودن و تأخیر (latency) نیست؛ یک **الزام امنیتی** است.

ردیابی مسیر شهودی

برای پاسخ به پرسش حیاتی «چرا عامل این کار را کرد؟»، تیم امنیت باید عدسی یکپارچه و زمان‌ترتیبی برای دیدن گام‌های شناختی عامل بسازد.

تجمیع سیگنال‌ها

با چارچوب‌های استاندارد تله‌متری، سیگنال‌های گوناگون — فراخوان‌های API، ورودی و خروجی ابزارها، بازیابی‌ها و تأخیر توکن — در یک **مسیر شهودی (Vibe Trajectory)** کامل جمع می‌شوند.

پویش محتوای متمرکز

در کنار گزارش‌برداری سنتی، بازرسی صریح همهٔ قطعه‌کدها و اسکریپت‌های پویایی که عامل در رانتایم می‌گیرد.

آنچه باید ثبت شود: جهش شناختی عظیم از پرامپت اولیهٔ کاربر تا درخت نحو انتزاعیِ کامپایل‌شده. این رد، حلقهٔ استدلال درونی عامل را به کنش‌های فیزیکی‌اش گره می‌زند و از ممیزی امنیتی شخص ثالث پشتیبانی می‌کند.

انحراف قصد و زوال اعتماد

چرا سیاههٔ ایستا کافی نیست: همان‌طور که عامل پویا منطق تولید می‌کند و ابزار تازه می‌گیرد، محیط امنیتی‌اش دائم نوسان می‌کند و سیاههٔ دارایی ایستا فوری کهنه می‌شود. به‌جایش باید صورت‌ریز **رانتایم** را پایش کرد: سندی زنده که شعاع اثر فعال عامل را در هر میلی‌ثانیه نگاشت می‌کند.

اصل زوال اعتماد (Trust Decay): اعتماد در سامانهٔ خودگردان دارایی مستهلک‌شدنی است. اعتماد وقتی از دست می‌رود که زنجیرهٔ تفکر درونی عامل زیرهدف‌هایی را دنبال کند که از قصد اولیهٔ انسان واگرا می‌شوند.

نمونه: پرامپت ساده‌ای مثل «پرس‌وجوی پایگاه داده را بهینه کن» ممکن است منحرف شود به تلاش عامل برای دانلود یک کتابخانهٔ نمایه‌سازی تازه و غیرمجاز.

نقاط بازگشت و قطع‌کنندهٔ باحالت

پیش از تغییر
سامانه یک نقطهٔ بازگشت در کنترل نسخه می‌سازد

←

حین اجرا
تحلیل رفتاری، مسیر را در برابر صورت‌ریز می‌سنجد

←

هر ناپایداری
امتیاز اعتماد پویای عامل را جریمه می‌کند

←

زیر آستانه
قطع‌کنندهٔ خودکار فعال می‌شود

و بازگشت با ظرافت انجام می‌شود: محیط از نقطهٔ بازگشت استفاده می‌کند تا تغییرها را فوری برگرداند، دسترسی ابزار را با ملایمت سلب کند و اجرای خودگردان را منجمد کند — **بدون خراب‌کردن `API`های متصل** و با حفظ وضعیت محیط برای تحلیل قانونی.

جمع‌بندی امنیت: پنج خط پایه

۱

حلقهٔ شهودی را سندباکس کنید

همیشه اسکریپت‌های تولیدشدهٔ پویا را در سندباکس سطح هسته و ایزوله از شبکه اجرا کنید؛ و تحلیل ترکیب نرم‌افزار به‌روز را برای یافتن وابستگی‌های توهمی یا آسیب‌پذیر تعبیه کنید.

۲

محیط را به چپ ببرید

منابع معتبر و فهرست‌های داخلی را الزامی کنید. مسدودسازی در ویرایشگر گام اول مشورتی است؛ اتکای اصلی بر وارسی‌های قطعی در چند نقطه از خط لوله است.

۳

اختیار محیطی صفر

هرگز «کلید سراسری» ندهید. هویت عامل‌محور و توکن‌های به‌هنگامِ به‌شدت محدود که با پایان کار منقضی می‌شوند؛ و برای کنش پرمخاطره، تفاوت‌نمای شهودی (Vibe Diff) به‌جای دکمهٔ کور.

۴

عملیات امنیتی عامل‌محور

پیوسته با عامل‌های تیم قرمز مجازی معماری‌تان را زیر فشار بگذارید، با تحلیل رفتاری صورت‌ریز پویا را پایش کنید و به تیم سبز اختیار بازسازی خودکار بدهید.

۵

مسیر اجرا را ردیابی کنید

فراخوان‌ها، ورودی ابزارها و گام‌های استدلال را ثبت کنید؛ پیوسته برای رفتار غیرمنتظره پایش کنید و از نقاط بازگشت کنترل نسخه برای سلب دسترسی هنگام انحراف استفاده کنید.

امنیت تضمین می‌کند عامل کار مخرب یا غیرمجاز نکرد. اما چطور قطعی ثابت کنیم که واقعاً قصد دقیق و چندلایهٔ کاربر را برآورده کرده است؟

عامل امن، لزوماً عامل مؤثر نیست. حالا باید جعبهٔ شیشه‌ای را باز کنیم.

بخش پنج

ارزیابی: هماهنگ‌کردن کیفیت در سامانه‌های قصدمحور

پرسشی که توسعه‌دهنده واقعاً دارد: آیا عامل همانی را ساخت که خواستم، و اصلاً خوب است؟

چرا ارزیابی این عامل‌ها متفاوت است

۱. شکاف کم‌مشخصگی

تست نرم‌افزار سنتی بر این فرض بی‌اغماض کار می‌کند که مشخصاتی کامل و سخت پیش از ارزیابی وجود دارد. کدنویسی شهودی (Vibe coding) دقیقاً برعکس است: «داشبورد را سریع‌تر کن» یک پروندهٔ تست نیست. نخستین کار ارزیابی این است که بسنجد آیا عامل این شکاف را پر کرد و مشخصات (spec) نانوشتهٔ درست را بازسازی کرد.

۲. کاربر اغلب نمی‌تواند خروجی را اعتبارسنجی کند

کاربر غیرفنی نمی‌تواند ۶۰۰ خط کد را خط‌به‌خط بازبینی کند. مهندس باتجربه هم در زمان واقعی نمی‌تواند. فاصلهٔ میان «عامل فکر می‌کند موفق شد» و «کد واقعاً درست است» اینجا از هر دستهٔ دیگری بیشتر است.

۳. نشست تکرارشونده است و کدبیس، وضعیت

هر نوبت (turn) فایل‌های واقعی را تغییر می‌دهد و تصمیم‌های بد اولیه انباشته می‌شوند. ارزیابی باید نه‌فقط تصمیم‌های تک‌نوبتی، بلکه کل قوس یک گفت‌وگوی چندنوبتی را روی کدبیسی زنده پوشش (coverage) دهد.

چه چیزی را بسنجیم — ابعاد کاربرمحور

۱

رضایت قصد

آیا عامل همانی را ساخت که کاربر **منظور داشت**، نه فقط آنچه گفت؟ سخت‌ترین بُعد برای سنجش، چون قصد نانوشته، مبهم و اغلب در میانهٔ نشست (session) متغیر است. و همان چیزی است که کاربر نهایتاً عامل را با آن قضاوت می‌کند.

۲

درستی کارکردی

آیا کد ساخته می‌شود، اجرا می‌شود و تست‌ها را پاس می‌کند؟ **کف است، نه سقف.** آسان برای سنجش و آسان برای تقلب: تست‌ها را می‌شود حذف یا جعل کرد تا قرمز سبز شود، بی‌آنکه چیزی درست شده باشد.

۳

درستی بصری و رفتاری

برای عامل‌هایی که برنامهٔ وب یا رابط کاربری می‌سازند، مصنوع همان خروجی رندرشده است، نه کد. سنجه‌های (rubric) سطح کد کاملاً نکته را از دست می‌دهند: صفحه یا درست به‌نظر می‌رسد و درست رفتار می‌کند، یا نه.

۴

هزینه و کارایی

مصرف توکن، تأخیر واقعی، تعداد فراخوان ابزار، و تعداد تکرار: کاربر چند بار باید اصلاح می‌کرد تا عامل هم‌گرا شود؟ عاملی که در یک نوبت به تغییر درست می‌رسد، محصولی کاملاً متفاوت از عاملی است که هشت اصلاح لازم دارد.

ابعاد درونی و محور عرضی ایمنی

۵

کیفیت کد و تطبیق با قراردادها

آیا کد با اصطلاحات، الگوها و قراردادهای پروژه می‌خواند؟ تغییری که تست‌ها را پاس می‌کند اما سبک کدبیس را نقض می‌کند، **خطای کدنویسی شهودی** است، حتی اگر در جای خودش درست باشد.

۶

کیفیت مسیر

آیا عامل مسیر معقولی رفت: اول فایل‌های مرتبط را خواند، ویرایش‌ها را منسجم مرتب کرد، و در هر گام ابزار یا مهارت (skill) درست را برگزید؟ **خروجی درست با استدلال بد، موفقیتی شکننده است.**

۷

رفتار خودترمیمی

وقتی ساخت شکست می‌خورد، تست شکست می‌خورد یا کاربر می‌گوید «نه، این‌طور نه»، آیا عامل بهبود می‌یابد یا خطاها را روی هم انباشته می‌کند؟ کیفیت بازیابی در طول یک نشست چندنوبتی روی هم انباشته می‌شود.

ایمنی و هوش مصنوعی مسئولانه، عرضی است: با چند بُعد تقاطع دارد — آسیب‌پذیری کد، رفتار امتناع، ایمنی محتوا، افشای مالکیت فکری — و باید **در کنار هر بُعد** سنجیده شود، نه به‌عنوان دروازه‌ای جدا.

ابعاد مستقل نیستند

کیفیت مسیر بهتر
بُعد ۶

←

درستی کارکردی بهتر
بُعد ۲

←

رضایت قصد
بُعد ۱ — پیش‌شرطش همان است

پیامد عملی: اگر رضایت قصد پایین است، پیش از دست‌کاری پرامپت‌ها، مسیر را بررسی کنید. اغلب ریشه در همان‌جاست: عامل فایل‌های مرتبط را نخوانده یا ترتیب ویرایش‌ها را غلط چیده است.

کارگاه ۱ — ۲۰ دقیقه

بازبینی امنیتی خروجی عامل

۱

اعتماد به سمت کاربر

آیا کلید، اعتبارسنجی گذرواژه یا پرچم نشست در سمت مرورگر رها شده است؟

۲

پشتیبان رهاشده

آیا کنترل دسترسی سطح ردیف و رد پیش‌فرض فعال است؟

۳

وابستگی‌ها

همهٔ بسته‌ها واقعاً وجود دارند؟ از منبع معتبرند؟ نسخه قفل شده؟

۴

دامنهٔ اختیار

عامل با چه هویتی اجرا کرد؟ کدام مجوزها لازم نبودند؟

معیار پذیرش: هر گروه باید دست‌کم یک یافتهٔ واقعی با **فایل، خط و اصلاح مشخص** بیاورد. «شاید بهتر باشد امن‌تر بنویسیم» یافته نیست.

چگونه بسنجیم — چهار روش نخست

روش	شرح	مناسب کدام بُعد
تست کارکردی خودکار	اجرای ساخت، مجموعه‌تست و لینترها روی خروجی عامل، وصل‌شده به خط لولهٔ پروژه. ارزان‌ترین سیگنال موجود.	درستی کارکردی؛ بخش‌های قاعده‌پذیر کیفیت کد
ارزیابی امنیت و ایمنی	ترکیب تحلیل ایستای امنیتی روی کد تولیدشده با کاوش خصمانه برای سنجش رفتار امتناع؛ پویشگرهای ایستا برای آسیب‌پذیری و ابزارهای یافتن نشت اعتبارنامه.	عرضی — کنار همهٔ ابعاد
داور مدل‌زبانی (LLM-as-Judge) یا عامل‌داور	نمره‌دهی خروجی در برابر سنجه‌ها، برای ابعادی که قاعده پاسخ درست را نمی‌گیرد. عامل‌داور می‌تواند رد اجرا (trace) را برای انسجام برنامه بازرسی کند.	رضایت قصد، کیفیت کد و سبک، کیفیت مسیر
تست مرورگرمحور	اجرای گردش‌کارهای چندگامی روی برنامهٔ مستقرشده و مشاهدهٔ نتیجه: اسکریپت‌های تعامل با رابط رندرشده و مقایسهٔ تصویر با مرجع.	درستی بصری و رفتاری

چگونه بسنجیم — سه روش دیگر

روش	شرح	مناسب کدام بُعد
بازرسی مسیر	تحلیل استدلال عامل، فراخوان ابزارها، فراخوانی مهارت‌ها و بازیابی‌ها. بستر کار، ردهای تله‌متری (telemetry) با دادهٔ فراخوان ابزار (tool call) در سطح دامنه است که با ابزارهای بازپخش رد نمایش داده می‌شوند.	کیفیت مسیر، رفتار خودترمیمی
بازبینی انسانی	نمونه‌گیری از نشست‌ها برای بازبینی مستقیم بازبینان واجد شرایط. مقیاس‌پذیر نیست؛ عمدتاً برای کالیبره‌کردن بقیهٔ روش‌ها به کار می‌رود.	رضایت قصد (تنها مرجع حقیقت)، کیفیت کد، قضاوت‌های دقیق ایمنی
ارزیابی برخط	نمونه‌گیری از ترافیک زندهٔ تولید و نمره‌دهی با همان سنجه‌های برون‌خط. همهٔ ابعاد را با نرخ نمونه‌گیری پوشش می‌دهد.	همه

ترفند نمونه‌گیری: نمونهٔ یکنواخت یک‌درصدی، دنبالهٔ بلند را از دست می‌دهد. سوگیری را به‌سمت نشست‌های پرهزینه، نشست‌های با اصلاح‌های متعدد، و نشست‌هایی که کاربر رهایشان کرده ببرید.

بنچمارک‌های استاندارد: مفید، اما با دام

چرا ارزشمندند

بنچمارک‌ها (benchmark) عامل شما را با میدان رقابت روی مجموعه‌کارهای مشترک مقایسه می‌کنند و قابلیت‌های شناختی خاص را از نویز محیط سفارشی سازمان جدا می‌کنند. مبنای تجربی لازم برای اعتماد به سامانه‌ای غیرقطعی را فراهم می‌کنند.

دام بیش‌برازش بنچمارک

عامل‌ها را می‌توان چنان بهینه کرد که روی مجموعه‌دادهٔ ایستا امتیاز درجه‌یک بگیرند اما در مواجهه با واقعیت آشفته و متناقض قصد انسانی در تولید، فاجعه‌بار شکست بخورند.

قاعدهٔ استفاده: امتیاز بالا در بنچمارک ریپوهای واقعی ثابت می‌کند عامل می‌تواند در ساختاری منظم حرکت کند، اما **هیچ تضمینی** نمی‌دهد که قضاوت زیبایی‌شناختی لازم برای ساخت یک برنامهٔ مصرفی را دارد. بنچمارک‌ها را صرفاً برای **کالیبراسیون شناختی** به کار ببرید، نه جایگزین ارزیابی قصد سفارشی.

رصدپذیری، پیش‌شرط ارزیابی است

برای ارزیابی استدلال درونی عامل، باید بتوانید ببینیدش. بدون رصدپذیری، خطاهای عامل به‌صورت رویدادهای یکپارچه و توضیح‌ناپذیر ظاهر می‌شوند.

ردیابی تفکر

دامنهٔ نشست کل مدت کار را پوشش می‌دهد، دامنهٔ تفکر چرخهٔ استدلال و پرامپت‌دهی پیش از کنش را ثبت می‌کند، و دامنهٔ ابزار آرگومان‌ها و تأخیر تعامل‌های محیطی را.

ردیابی هزینه

با تجمیع ویژگی‌های دامنه‌ها، تیم‌ها می‌توانند مصرف توکن، تأخیر استنتاج و هزینهٔ حلقه‌های (loop) خودترمیمی را دقیق اندازه بگیرند.

نمونه‌گیری دم‌محور پویا

گرفتن صددرصد ردها در تولید، بودجهٔ ذخیره‌سازی را می‌بلعد. نمونه‌گیری دم‌محور اجازه می‌دهد جمع‌کننده پس از پایان، کل رد را ارزیابی کند و موفقیت‌های روزمره را دور بریزد و ردهای دارای خطا یا حلقهٔ خودترمیمی افراطی را نگه دارد.

نکتهٔ کاربردی ۱ — پیشوند نشست را سنجهٔ قصد کنید

کدنویسی شهودی مشخصاتی برای آزمودن ندارد؛ قصد کاربر نانوشته است و در طول نوبت‌ها تکامل می‌یابد. نزدیک‌ترین چیز به مشخصات، یک یا دو پیام نخست کاربر است.

```
# Derive acceptance criteria from the user's opening turns
opening = " ".join(session.user_messages[:2])
criteria = client.models.generate_content(
    model="...",
    contents=f"Produce 3-5 acceptance criteria for: {opening}. Return JSON.",
).parsed["criteria"]

# Score every agent turn against the derived criteria
score = client.models.generate_content(
    model="...",
    contents=f"Does this output satisfy {criteria}? Score 1-5 with rationale."
             f"Output: {agent_response}",
).parsed
```

چرا مهم است: این تنها راه عملی برای سنجش رضایت قصد در مقیاس است.

نکتهٔ کاربردی ۲ — مصنوع رندرشده را داوری کنید، نه کد را

در کدنویسی شهودی، کاربر خروجی را قضاوت می‌کند نه تغییرات کد را. مدلی چندوجهی که به صفحهٔ رندرشده نگاه می‌کند، مشکلاتی را می‌گیرد که ارزیابی سطح کد کاملاً از دست می‌دهد.

چیدمان به‌هم‌ریخته در موبایل

کد بی‌نقص، تجربه خراب.

کنتراست ناکافی

مشکل دسترس‌پذیری که هیچ تست واحدی نمی‌گیرد.

وضعیت اشتباه دکمه‌ها

رفتار تعاملی نادرست.

ترکیب درست: داور چندوجهی مسائل بصری و طراحی را می‌گیرد؛ ادعاهای تست مرورگرمحور، تعامل‌های خراب را می‌گیرند.

نکتهٔ کاربردی ۳ — هم‌گرایی نشست را بسنجید، نه درستی تک‌نوبتی

نشست کدنویسی شهودی ذاتاً چندنوبتی است. پرسش مربوط این نیست که «آیا نوبت چهارم درست بود؟» بلکه این است که «آیا کاربر به چیزی که می‌خواست هم‌گرا شد؟»

```
def session_outcome(trace):
    return {
        "converged":         trace.last_turn.user_signal == "satisfied",
        "turns_to_converge": trace.user_correction_count,
        "abandoned":         trace.last_user_action == "close",
        "cost_to_converge":  trace.total_token_cost_usd,
    }
```

آموزنده‌ترین خطاها: نشست‌هایی که کاربر در میانه رهایشان کرده — به‌مراتب آموزنده‌تر از خطاهای تک‌نوبتی.

نکتهٔ کاربردی ۴ — اصلاح‌های کاربر، دادهٔ برچسب‌خوردهٔ رایگان‌اند

هر «نه، این‌طور نه» از سوی کاربر، یک نمونهٔ خطای برچسب‌خورده است — و کدنویسی شهودی این‌ها را به‌وفور تولید می‌کند.

جمع‌آوری
همهٔ نوبت‌های اصلاحی از ردها

←

جاسازی
تبدیل به بردار

←

خوشه‌بندی
گروه‌بندی معنایی

←

اولویت‌بندی
خوشه‌ها = فهرست حالت‌های شکست برای تکرار بعدی

و مزیت اصلی: شکاف‌های نظام‌مند عامل بسیار سریع‌تر از ساختن یک بنچمارک خطای مصنوعی دیده می‌شوند.

جمع‌بندی و تکلیف

تز جلسه: مدل خام صرفاً یک موتور است؛ فقط وقتی به عامل آمادهٔ سازمان بدل می‌شود که در مهاری مستحکم پیچیده شود. امنیت فقط ثابت می‌کند عامل آسیبی نزد. با جفت‌کردن آن با چارچوب ارزیابی دقیق، می‌توان مطمئن ثابت کرد که عامل واقعاً ارزش تولید کرده است.

1. یک گاردریل (guardrail) قطعی در ریپو (repository) خودتان فعال کنید که دست‌کم یکی از دو الگوی خطا را بگیرد.

   قلاب پیش از کامیت (commit) یا وارسی خط لوله.

2. ده نشست اخیرتان را مرور و اصلاح‌های خودتان را دسته‌بندی کنید.

   سه حالت شکست نظام‌مند را نام ببرید.

3. برای پرمخاطره‌ترین گردش‌کار عاملی پروژه‌تان، طرح مهار مخاطره بنویسید.

   سندباکس، دامنهٔ هویت، نقاط تأیید انسانی، سازوکار بازگشت.

جلسهٔ بعد – پایانی

توسعهٔ مبتنی بر مشخصات در سطح تولید

همه‌چیز را در یک گردش‌کار واقعی جمع می‌کنیم

تولید تا حد زیادی حل شده است. راستی‌آزمایی، امنیت و قضاوت معماری، پیشهٔ تازه‌اند.